Assignment 9

Advanced SQL Queries Using Joins and Subqueries

Multi-table reports, correlated subqueries, EXISTS, and summary statistics

Project: LibSphere - Library Management System **Prepared:** 23 October 2026

Platform assumption: MySQL 8.0.16 or later (InnoDB). Outputs in this report are expected results derived from the included sample dataset; run the SQL in MySQL Workbench to capture authentic execution screenshots if required by the instructor.

These reports use the same finalized schema and sample records. Joins combine related entities for readable reports; subqueries answer conditions that depend on other query results.

Report 1 - Current open-loan register (INNER JOIN)

```
-- Report 1: Open-loan register using INNER JOINs
SELECT l.loan_id, b.title,
       CONCAT(m.first_name, ' ', m.last_name) AS member_name,
       l.loan_date, l.due_date, l.status
FROM Loans AS l
JOIN Books AS b ON b.book_id = l.book_id
JOIN Members AS m ON m.member_id = l.member_id
WHERE l.status IN ('Issued', 'Overdue')
ORDER BY l.due_date, l.loan_id;
```

loan_id	title	member_name	loan_date	due_date	status
1	Clean Code	Asha Rao	2026-10-01	2026-10-15	Overdue
3	The Pragmatic Programmer	Priya Sharma	2026-10-05	2026-10-19	Overdue
4	Nineteen Eighty-Four	Karthik Iyer	2026-10-10	2026-10-24	Issued
2	Clean Code	Rahul Menon	2026-10-20	2026-11-03	Issued
5	Pride and Prejudice	Meera Nair	2026-10-21	2026-11-04	Issued
6	Computer Networks	Asha Rao	2026-10-22	2026-11-05	Issued

Explanation: INNER JOIN returns only loans with matching book and member records; WHERE excludes returned transactions.

Report 2 - Book catalogue with author(s) and category

```
-- Report 2: List each title, its category, and its author(s)
SELECT b.book_id, b.title, c.category_name,
       GROUP_CONCAT(CONCAT(a.first_name, ' ', a.last_name)
                    ORDER BY a.last_name SEPARATOR ', ') AS author_list
FROM Books AS b
JOIN Categories AS c ON c.category_id = b.category_id
LEFT JOIN Book_Authors AS ba ON ba.book_id = b.book_id
LEFT JOIN Authors AS a ON a.author_id = ba.author_id
GROUP BY b.book_id, b.title, c.category_name
ORDER BY b.book_id;
```

book_id	title	category_name	author_list
1	Clean Code	Computer Science	Robert C. Martin
2	The Pragmatic Programmer	Computer Science	Andrew Hunt, David Thomas
3	Nineteen Eighty-Four	Fiction	George Orwell
4	Sapiens	History	Yuval Noah Harari
5	Pride and Prejudice	Fiction	Jane Austen
6	Computer Networks	Computer Science	Andrew S. Tanenbaum
7	The Mythical Man-Month	Business	Frederick P. Brooks Jr.
8	A Brief History of Time	Science	Stephen Hawking

Explanation: The bridge table resolves the many-to-many relationship. GROUP_CONCAT combines multiple author names into one readable catalogue field.

Report 3 - Titles with more than one recorded loan (correlated subquery)

```
-- Report 3: Titles with more than one recorded loan (correlated subquery)
SELECT b.title,
       (SELECT COUNT(*) FROM Loans AS l WHERE l.book_id = b.book_id) AS loan_count
FROM Books AS b
WHERE (SELECT COUNT(*) FROM Loans AS l WHERE l.book_id = b.book_id) > 1
ORDER BY loan_count DESC, b.title;
```

title	loan_count
Clean Code	3
Computer Networks	2
Nineteen Eighty-Four	2
Pride and Prejudice	2
The Pragmatic Programmer	2

Explanation: For each book row, the correlated subquery counts Loans rows that match that book_id. The outer WHERE retains counts greater than one.

Report 4 - Members with overdue loans (EXISTS)

```
-- Report 4: Members who have at least one overdue loan (EXISTS subquery)
SELECT m.member_id, CONCAT(m.first_name, ' ', m.last_name) AS member_name
FROM Members AS m
WHERE EXISTS (
    SELECT 1 FROM Loans AS l
    WHERE l.member_id = m.member_id AND l.status = 'Overdue'
)
ORDER BY m.member_id;
```

member_id	member_name
1	Asha Rao
3	Priya Sharma

Explanation: EXISTS is true when at least one overdue transaction exists for a member; it avoids duplicating a member if that member has multiple overdue loans.

Report 5 - Transactions by category (LEFT JOIN and aggregation)

```
-- Report 5: Number of recorded loan transactions by category
SELECT c.category_name, COUNT(l.loan_id) AS loan_transactions
FROM Categories AS c
LEFT JOIN Books AS b ON b.category_id = c.category_id
LEFT JOIN Loans AS l ON l.book_id = b.book_id
GROUP BY c.category_id, c.category_name
ORDER BY loan_transactions DESC, c.category_name;
```

category_name	loan_transactions
Computer Science	7
Fiction	4
Business	1
History	1
Science	1

Explanation: LEFT JOIN retains every category, including one with no recorded loans; COUNT(l.loan_id) counts transactions rather than category or book rows.

Report 6 - Members with above-average borrowing activity (subquery)

```
-- Report 6: Members whose recorded loan count is above the per-member average
SELECT m.member_id,
       CONCAT(m.first_name, ' ', m.last_name) AS member_name,
       COUNT(l.loan_id) AS loan_count
FROM Members AS m
JOIN Loans AS l ON l.member_id = m.member_id
GROUP BY m.member_id, m.first_name, m.last_name
HAVING COUNT(l.loan_id) > (
    SELECT AVG(member_counts.loan_count)
    FROM (
        SELECT COUNT(*) AS loan_count
        FROM Loans
        GROUP BY member_id
    ) AS member_counts
)
```

```
ORDER BY loan_count DESC, m.member_id;
```

member_id	member_name	loan_count
1	Asha Rao	3
3	Priya Sharma	3
4	Karthik Iyer	3

Explanation: The nested query calculates the average number of recorded loans per member (14 / 6 = 2.3333). The HAVING clause retains members with more than that average.

Report 7 - Inventory availability

```
-- Report 7: Inventory view - copies currently unavailable for checkout
SELECT title, total_copies, available_copies,
       (total_copies - available_copies) AS copies_not_available
FROM Books
ORDER BY book_id;
```

title	total_copies	available_copies	copies_not_available
Clean Code	5	3	2
The Pragmatic Programmer	4	3	1
Nineteen Eighty-Four	6	5	1
Sapiens	4	4	0
Pride and Prejudice	3	2	1
Computer Networks	5	4	1
The Mythical Man-Month	3	3	0
A Brief History of Time	4	4	0

Explanation: total_copies minus available_copies estimates copies checked out now. This relies on the operational rule that available_copies is updated correctly in the same transaction as an issue/return.

Report 8 - Books with above-average stock (scalar subquery)

```
-- Report 8: Titles whose stock exceeds the average stock per title
SELECT title, total_copies
FROM Books
WHERE total_copies > (SELECT AVG(total_copies) FROM Books)
ORDER BY total_copies DESC, title;
```

title	total_copies
Nineteen Eighty-Four	6
Clean Code	5

title	total_copies
Computer Networks	5

Explanation: the scalar subquery returns the average stock value (4.25); the outer query selects titles whose total_copies are greater than the average.

Conclusion: Joins provide combined reports from related tables, while correlated and scalar subqueries support existence checks and comparisons against calculated values. For submission, run the queries in MySQL Workbench and add screenshots of the actual result grids if your course specifically requires screenshots.